

Tacit: Confidential Assets and DeFi on Bitcoin, Bridged by Zero-Knowledge Reflection

z0r0z
<https://tacit.finance>

Version 1 2026

Abstract

Bitcoin can hold value without a trusted party, but issuing assets on it, hiding amounts, or trading them has meant accepting one: an off-chain proof courier, a federation, a rollup operator, or a custodian. Tacit removes that party in two steps.

First, it is an indexer-validated metaprotocol. Assets live in Taproot envelopes, amounts are Pedersen commitments with range proofs, and conservation is a Schnorr signature over the excess. Any indexer running the specification reaches the same state from the chain alone.

Second, Tacit reflects that state into an immutable confidential pool on Ethereum. Every pool settlement is one SP1 zero-knowledge proof. Bitcoin blocks reach the pool through a full-proof-of-work header relay and an SP1 guest that folds Tacit envelopes. Pool state returns to Bitcoin through a light-client proof verified recursively inside the same guest. No signer stands between the chains.

On this base the pool runs confidential DeFi:

- one-transaction entry from ETH or any ERC-20, and exit back to public tokens;
- AMM swaps and routes, with a prover-blind batch op backed by a ceremony Groth16 circuit;
- OTC trades, bids and adaptor-signature swaps;
- stealth payments;
- a CDP issuing cUSD;
- cBTC, minted against real BTC in self-custody locks;
- farms;
- gasless relaying with the fee bound inside the proof.

Every balance recovers from a private key and public chain data. The core is immutable, yet the protocol keeps improving: it grows at the edges, and each pool can deploy its own successor, which inherits the Bitcoin lane without a gap. Bytes are reserved for a V2 built on Bitcoin covenants, which makes cBTC fully trustless, and for a shielded pool native to Bitcoin that cBTC feeds.

1 Introduction

A useful asset layer on Bitcoin needs five properties:

1. its state is public data anyone can check;
2. amounts are hidden;
3. value can move to where computation is cheap without trusting a bridge;
4. BTC itself can back a fungible asset;
5. a user can recover everything from a key.

Existing designs each give up one:

Design	What it gives up
Ordinals, Runes	Amounts are public.
RGB, Taproot Assets	Validity moves off-chain, so losing a proof means losing the balance.
Liquid	Relies on a federation.
Rollups and Bitcoin L2s	Rely on an operator set.
Wrapped BTC	Relies on a custodian or a threshold group.

Tacit provides all five by composing existing systems rather than adding consensus. Bitcoin orders the data and indexers read it (§2). Commitments hide amounts (§3). An immutable contract on Ethereum accepts only state transitions that carry a zero-knowledge proof (§4). The chains learn about each other through proofs, not signatures (§5). Real BTC backs cBTC through self-custody locks, secured economically until Bitcoin gains covenants (§6, §11). Every balance recovers from a key (§7).

2 A metaprotocol on Bitcoin

A Tacit op is a commit/reveal pair. The commit pays to a Taproot output whose only script leaf is:

$\langle P \rangle$ OP_CHECKSIG OP_FALSE OP_IF "TACIT" 0x01 $\langle payload \rangle$ OP_ENDIF

The reveal spends the commit by script path, so the payload rides in witness data. Bitcoin nodes do not interpret it. Indexers do: a Tacit UTXO is valid iff the op that created it passes its rules and each of its asset inputs is itself valid.

The rules are deterministic, so two indexers that see the same bytes reach the same verdict. There is no quorum, no vote and no leader. Ordinals and Runes use the same trust model. What users trust is the published specification and its open reference implementations.

The Bitcoin reflection guest (§5) is a second, independent check. It re-implements the rules for every op it folds, and a verifying key that cannot be changed fixes its verdict.

Ops cover:

- **Issuance:** confidential supply, or fair-launch mints against a public cap. TAC, the native asset, was issued with T_CETCH and a zero mint authority, so its supply is fixed at 21 million.
- **Transfers:** hidden amounts, with optional stealth addresses.
- **Trades:** atomic OTC trades against BTC, bids that a watchtower fills while the buyer is offline, and a native AMM with per-trade swaps, routes, uniform-price batch clearing and LP farms. Its first pool, tETH/TAC, was founded from Bitcoin-side notes and is live in the indexer.
- **cBTC:** locks and redeems.
- **Cross-chain:** bridge burns, re-mints of notes crossing back from Ethereum, and value-free calls in both directions.

An unknown opcode creates no state, so the protocol grows by adding opcodes.

3 Confidential value

Every amount is a Pedersen commitment on secp256k1:

$$C = v \cdot H + r \cdot G,$$

where G is Bitcoin’s generator and H is a nothing-up-my-sleeve point hashed by SHA-256 from the tag `tacit-generator-H-v1`. Commitments hide the value perfectly and add homomorphically. A transaction balances iff its excess

$$E = \sum C_{\text{out}} + b \cdot H - \sum C_{\text{in}}$$

carries no H component, where b is any public burn. Tacit proves this with a BIP-340 signature under E , a Mimblewimble-style kernel. Only someone who knows every blinding can sign, and nobody can sign for an excess containing value.

Range proofs bound every output to $[0, 2^{64})$, aggregated over up to eight outputs. New transfers use Bulletproofs+, and classic Bulletproofs remain accepted. The recipient’s blinding and an 8-byte amount keystream come from an ECDH shared secret between sender and recipient, anchored to the transaction’s first asset input. A recipient therefore finds and opens every credit from its own key and the chain: no share link, no sync server.

The same commitment, with the same G and H , is the note in the Ethereum pool. One note format on both chains lets value cross between them without changing representation.

4 The confidential pool

The pool is one immutable contract holding three depth-32 keccak Merkle trees: notes, locks and CDP positions. Each tree has its own spent set, and one-shot guards ensure that a bridge burn, a cBTC lock, a farm harvest and a Bitcoin source pay out at most once. The pool also escrows every external asset and ETH, and holds the reflected state of Bitcoin. It has no owner and no pause.

A note is $(\text{asset}, C, \text{owner})$. A pool-native note’s leaf is a keccak hash of those fields. A Bitcoin-homed note’s leaf commits instead to the note’s Bitcoin key and the deployment it is bound to. Every nullifier is derived from the leaf. For an owned note the nullifier also takes the owner’s secret nullifier key, so an observer cannot tell which leaf a spend consumes. Bearer and Bitcoin-homed notes use a nullifier derived from the leaf alone, which anyone can compute.

Entry and exit. `wrap` escrows ETH or any ERC-20 and records a pending deposit that only its depositor’s proof can consume; one transaction can wrap and immediately spend it as a transfer, swap, liquidity add or CDP mint. Assets the pool itself mints — TAC, cBTC, cUSD — need no escrow: unwrapping mints the canonical ERC-20, and wrapping it back burns it. A note can also exit partway, as a public payout plus hidden change.

Every change to notes, locks and positions is a batch of up to 256 ops proven by one SP1 program. Per op, the program checks:

- that each input leaf is a member of a known root, without revealing which leaf;
- the range proofs;
- the kernel or opening proofs;
- the op’s own rules.

It then commits the public effects: nullifiers, new leaves, payouts, fees, swaps and liquidity changes, bridge claims and memos. The contract verifies the proof against a pinned verifying key and applies exactly those effects.

A spend proves membership against any root the tree has ever had. The anonymity set of an owned-note spend is therefore every owned note of its asset.

Nothing in the pool accumulates cost with use:

- an insert is 32 hashes at any fill;
- nullifier and root sets are constant-time maps that are never iterated;
- the guest touches history only through fixed-depth proofs.

The contract also enforces a floor against inflation: the pool never records more spent native notes than it has ever created.

The pool binds every proof to $CHAIN_BINDING = \text{keccak}(\text{chainid} \parallel \text{address})$, so a proof for one deployment is invalid in any other.

5 Reflection: a bridge without signers

Bitcoin to Ethereum. Anyone may extend `BitcoinLightRelay`, a header relay that checks full proof-of-work and follows the heaviest chain. The reflection guest proves a run of blocks on top of it. For each block it:

1. re-hashes every transaction to the header’s Merkle root;
2. nullifies every input that spends a reflected note;
3. re-verifies and folds each Tacit envelope it understands, and skips the rest;
4. extends a digest chain over the result.

The pool accepts an attestation only if its prior digest matches, its tip is buried at least 24 blocks under the relay tip, and its counters match the pool’s own. The pool keeps every attested root of Bitcoin’s Tacit notes, so a proof against one never goes stale. It also holds the current roots of Bitcoin’s spent set and bridge burns, cBTC lock records, and pending Bitcoin calls.

Value enters the pool from Bitcoin in two ways:

- **Bridge burn.** The note is destroyed on Bitcoin under a burn id that commits to the target deployment, and the pool mints it exactly once.
- **Fast lane.** A Bitcoin-homed note bound to this deployment (by `T_CXFER_BOUND`) is spent directly on Ethereum. The proof must be made against the current reflected Bitcoin spent root, show that the note’s nullifier is absent from it, and carry a signature from the note’s Bitcoin key. The pool records the consumed source so that reflection retires the note on Bitcoin.

Ethereum to Bitcoin. A second guest, built on the `sp1-helios` sync-committee light client, proves finalized pool storage:

- cross-out commitments,
- consumed sources,
- outbox messages.

The committee chains from period to period, with each step committed. The Bitcoin reflection guest verifies this proof recursively and folds, by membership:

- re-mints of crossed-out notes on Bitcoin;
- outbox messages;
- retirements of fast-lane notes.

Calls. A value-free Bitcoin envelope can authorize an Ethereum call, which an executor contract runs once it is reflected. Ethereum can post messages that reflection delivers to Bitcoin.

The bridge’s soundness rests on Bitcoin proof-of-work, the Ethereum sync committee and SP1. There is no multisig, attester set or wrapped-asset IOU. Its liveness needs someone to run the provers. Anyone may, and Ethereum-homed notes stay spendable if nobody does.

A full round trip has settled on mainnet: a note crossed out of the pool, was re-minted on Bitcoin, burned there and minted back, proven in both directions.

6 Confidential DeFi

Swaps. Pools are constant-product curves with a fee tier and an optional protocol-fee switch. Reserves are public. A plaintext AMM contract, `TacitPublicAmm`, trades against the same reserves, so public and confidential liquidity share one curve.

- `OP_SWAP_ROUTE` swaps a hidden input note through up to four pools, with an end-to-end minimum output. The dapp settles every swap this way; a single swap is a one-hop route.
- `OP_SWAP` clears many hidden-output intents at one uniform price. Traders in one batch pay the same price, so they cannot sandwich each other. The dapp offers it as an opt-in batch.

Each swap changes public reserves, so a swap settled alone shows its size. A batch shows only its net change.

Prover-blind swaps. Whoever proves an `OP_SWAP` or `OP_SWAP_ROUTE` sees its amounts. `OP_SWAP_BLIND` keeps them out of the SP1 witness. The batcher that computes the clearing and produces its Groth16 proof still sees them; the SP1 prover and the chain do not. The op is live in the guest; swaps route through `OP_SWAP` or `OP_SWAP_ROUTE` until the relay’s batch coordinator ships.

Each trader commits to its input on BabyJubJub, the curve native to BN254. A 169-byte cross-curve sigma proves that commitment equals the trader’s secp256k1 note. The batch carries one Groth16 proof of the `amm_swap_batch` circuit, which proves over hidden amounts that:

- the clearing price is correct;
- every fill is correct;
- every minimum output is met.

The circuit’s verifying key comes from the AMM trusted-setup ceremony: Hermes Phase 1, then a Phase 2 of 5,018 contributions sealed by a Bitcoin-block beacon. It is compiled into both SP1 guests, which verify the pairing in-guest. The reflection guest uses the same key for `T_SWAP_BATCH` on Bitcoin.

A circuit fits here because uniform-price clearing over hidden amounts is cheap as a circuit and expensive as a zkVM program. The guests’ own verifying keys fix the circuit key.

Trades without a pool.

- `OP_UTC` swaps two parties’ notes directly.
- `OP_BID` is a limit order with a pre-signed grid of partial fills, so the buyer need not be online.
- Adaptor locks tie a claim to the revelation of a Schnorr signature scalar, which gives atomic swaps against the other chain.

Stealth payments. A payer locks a note under a one-time key derived from the recipient’s published key. The recipient claims it with a signature, and the payer may refund after a deadline. A Bitcoin bridge burn can mint straight into such a lock.

cUSD. A CDP locks a basket of cBTC notes and mints cUSD. The position’s owner is hidden, though the controller prices its amounts against a BTC/USD feed in the open. Minting requires 150% collateral today; a position becomes liquidatable below 130%, and liquidation repays the debt and seizes the basket. A stability fee and a savings rate are built in and dormant. Policy lives in a governed **CollateralEngine** that the pool only ever accepts or rejects, with on-chain caps on its ratios and fee, and a cooling-off period before a feed change or a stricter liquidation ratio can bite.

cBTC. A user locks BTC on Bitcoin with a **T_CBTC_LOCK** envelope that pre-commits to the note it will mint. Reflection records the lock’s value, and the pool mints cBTC once per lock, held by the locker’s own key. Anyone may fund a wstETH escrow of at least $1.5\times$ the lock’s value, which makes spending the lock unprofitable: a minted lock spent without a matching redeem is visible to reflection and slashable into the insurance reserve, and each funder reclaims its share after a proven redeem or before any mint. The amount minted is oracle-free — a price feed only sizes the escrow, so a stale feed can pause a mint but never inflate one. cBTC is live on mainnet as a pilot. Custody here is economic, not enforced; covenants close that gap (§11).

cBTC is a Tacit asset on both chains. A cBTC note crosses out of the pool with **OP_BRIDGE_BURN** and is re-minted on Bitcoin by **T_CROSSOUT_MINT**, proven by the light client of §5. There it moves with hidden amounts like any other Tacit note. This is how BTC reaches Bitcoin-native privacy (§11).

Farms. LP-share notes bond into a receipt note; a farm controller tracks reward-per-share at entry, pays a harvest from its own escrowed treasury, and returns the shares on unbonding. The live controller, **FarmManager**, streams wTAC — an ownerless 1:1 wrapper of TAC — across three LP pools, under a governor that can reweight pools within capped, delayed bounds but can never shorten a running stream or lower its total rate.

7 Gasless, self-sovereign, recoverable

Any op may be relayed. The relayer’s fee is part of what the proof commits to, bound by the conservation kernel, the opening proof, or the note’s spend signature. The relayer pays gas and collects exactly that fee. It cannot redirect a payout, raise the fee, or submit the proof after a deadline the user sets. A relay fee is zero or has at most two significant digits.

Once value is in the pool, a user never needs ETH to move it. Entry is one Ethereum transaction, and a tip forwarder lets that same transaction pay a relayer to prove and settle the deposit, so the user runs no prover at all.

The relayer is optional. **settle** is open to anyone, proofs can be generated locally or on an open prover network, and the three guest programs rebuild byte for byte from source. A user can prove locally, settle directly, and involve no third party.

Recoverable. A wallet rebuilds every balance and position from its key and public chain data. Bitcoin credits are found by trial derivation against each input’s sender key (§3). Pool notes carry memos encrypted to their owner and committed in the proof that created them, and positions are read back from the pool’s events. There is no backup file to lose and no server to ask.

8 An immutable core that keeps improving

The pool, its guests and their verifying keys cannot be upgraded, so nobody can change the rules under a note that already exists. The protocol still grows, in two ways that never touch the core.

At the edges. New Bitcoin opcodes are additive: an unknown opcode creates no state, so indexers adopt new ones without disturbing the old. Anyone can register an ERC-20 into the pool. New debt assets and reward programs plug in as controllers that the pool consults only to accept or reject. Clients, relayers and provers improve freely, because each only produces proofs the fixed guests already verify; the blind-swap coordinator of §6 is one such upgrade. V1 gains depth and features as its market grows, with no migration at all.

By succession. The pool live today is the root of the V1 lineage. Each pool deploys its own successor through `createNextGen`, its one privileged call, which the ops multisig holds. It runs once, from the pool’s own address, and a successor accepts a predecessor only if that predecessor deployed it. The steward chooses the successor’s code and nothing else: it cannot touch escrow, freeze an exit or redirect a payout. Succession carries state forward instead of resetting it:

- **The Bitcoin lane continues without a gap.** The successor’s first reflection proof rebases on the predecessor’s attested digest, tip and counters, read live from the predecessor’s contract. The new pool picks up Bitcoin’s reflected state exactly where the old one left off, so value held on Bitcoin never has to move.
- **Assets keep their identity.** Asset ids are shared across chains and generations, so TAC, tETH and every Bitcoin asset are the same asset in each pool.
- **The old pool keeps its promises.** It refuses new value, but every exit, every release of value already committed, and its reflection stay open. A burn that targeted it still pays, and a cBTC lock registered there still sees its redemption, which returns its escrow.
- **Users move on their own schedule,** by exiting one pool and entering the next. Nothing moves value on their behalf, and a pool deployed by anyone else is an isolated system that cannot touch the lineage.

Toward V2. V1 is built to hand off to a covenant generation. The covenant constructions of §11 need new Bitcoin opcodes, which indexers adopt additively, and new settle logic, for which V1 keeps pool op 5 (`OP_COVENANT_MINT`) and op codes 35–255 free. When Bitcoin activates a covenant, V2 deploys through V1’s own `createNextGen` and inherits the Bitcoin lane. cBTC then moves from economic to enforced custody one lock at a time: a locker redeems a V1 lock, which releases its escrow, and re-locks the sats in a covenant vault that mints in V2 with no escrow at all. Every V1 note keeps its exits throughout.

9 Privacy

	Hidden	Public
Bitcoin	amounts; stealth recipients; trade sizes in <code>T_SWAP_BATCH</code>	addresses, the transaction graph, asset ids, burn amounts, AMM reserves
Pool	amounts; which owned note a spend consumes	the deposit and withdrawal boundary, AMM reserves and their changes, CDP amounts
Relay	your key, seed and blindings	the witness of the ops it proves

The pool's anonymity set is every owned note of an asset, not one denomination. `OP_SWAP_BLIND` hides trade sizes from the SP1 prover too, once the relay's batch coordinator ships. Proving locally keeps a witness on one device end to end.

10 Trust and limits

Relied on:

- Bitcoin and Ethereum consensus;
- the soundness of SP1 and of Groth16;
- the sp1-helios sync committee;
- for the circuits in §6 only, at least one honest contributor to each ceremony;
- the price feeds of the **CollateralEngine**: BTC/USD for cUSD, and BTC per wstETH for cBTC escrow;
- the ops multisig (2-of-4, with a one-hour delay unless all four sign), within the on-chain bounds of the collateral and farm controllers;
- for a walk-away bid on Bitcoin only, a hosted watchtower holding a capped, dedicated key over the sats a buyer commits.

Not relied on: relayers, the hosted API, gateways and explorers. The reference indexer reads Bitcoin from public Esplora endpoints with fallbacks, and any other source gives the same state.

Limits:

- Reflection halts, rather than rewrites, on a Bitcoin reorg deeper than its confirmation depth.
- Anyone can submit a relayed proof first and collect its fee. The user's outcome does not change. This follows from a proof that works for any relayer.
- cBTC is economically secured, not custodially guaranteed.
- On-chain Bitcoin transfers carry a large witness. Tacit on Bitcoin is suited to settlement, and the pool is where frequent activity belongs.

11 Bitcoin covenants and the road ahead

Several constructions would be fully trustless if Bitcoin could restrict where an output may be spent. Tacit reserves room for them now:

- **Covenant cBTC.** Pool op 5, `OP_COVENANT_MINT`, is held for cBTC minted against a lock whose sats can leave only through a redemption. It would reuse the existing mint's value binding with no escrow, so cBTC would need no economic assumption.
- **On-chain bid escrow.** A covenant could bind a buyer's pre-signed input to its bid template. That would replace the watchtower that fills today's offline bids, and it would activate the reserved batch-fill and both-sides match bytes.
- **Fractional slots.** Reserved opcodes would split a BTC slot into fungible shares and recombine them, with no shared vault.

Any of CTV, `OP_CAT`, `OP_CHECKSIGFROMSTACK`, `OP_VAULT` or an equivalent suffices. Each ships as additive Bitcoin opcodes and in V2, the covenant generation of the pool, which V1 deploys and hands its Bitcoin lane to (§8). None changes what is already live.

A shielded pool on Bitcoin. Bitcoin opcodes `0x6C` (`T_BTC_SHIELD`) and `0x6D` (`T_BTC_SPEND`)

are reserved for a shielded pool that lives on Bitcoin alone, described in the companion paper *Secret Sats*. It holds Tacit asset value, not BTC, so it adds no custody:

- a shield spends transparent notes into a pool leaf;
- a spend proves membership, nullifiers, its owner’s signature and conservation in one SP1 proof, so it unlinks sender from receiver as well as hiding the amount;
- an exit returns value to an ordinary Tacit note.

Indexers replay the pool from Bitcoin and verify each proof locally, with no Ethereum dependency. Relayers post spends and are paid from a pool output, so a sender needs no Bitcoin wallet. cBTC is its bridge to BTC: sats locked on Bitcoin mint cBTC in the Ethereum pool (§6), cBTC crosses back to Bitcoin by light-client proof, and there it shields into the pool. Shield, private payment and exit run end to end on signet today. Sats will also move in and out by atomic swap, and by covenant once Bitcoin allows one.

Cryptography changes the same way: a better range proof is added alongside the old one, as Bulletproofs+ was. Proof systems verifiable in Bitcoin script could move parts of indexer validation into consensus.

12 Conclusion

Tacit builds an asset layer on Bitcoin without a bridge signer or custodian:

- a deterministic metaprotocol with hidden amounts;
- an immutable, proof-gated pool that carries the same notes into confidential DeFi;
- a bridge made of proofs in both directions;
- cBTC, which carries real BTC into that DeFi and back to Bitcoin’s own shielded pool;
- a lineage of immutable pools, each deployed by the last, that improves as its market grows and hands off to V2 when Bitcoin gains covenants.

Where cryptography can enforce a property, it does. Where Bitcoin cannot yet enforce one, the gap is stated, bounded, and reserved for a covenant to close.

References

1. Rodarmor, C. *Ordinals* (2023); *Runes* (2024).
2. Pedersen, T.P. *Non-interactive and information-theoretic secure verifiable secret sharing*. CRYPTO 1991.
3. Jedusor, T.E. *Mimblewimble* (2016); Poelstra, A. *Mimblewimble* (2016).
4. Bünz, B. et al. *Bulletproofs*. IEEE S&P 2018. Chung, H. et al. *Bulletproofs+*. 2020.
5. Wuille, P.; Nick, J.; Ruffing, T.; Towns, A. *BIP-340*, *BIP-341*. 2020.
6. Groth, J. *On the size of pairing-based non-interactive arguments*. EUROCRYPT 2016.
7. Succinct. *SP1 zkVM*; *sp1-helios*.
8. Pertsev, A.; Semenov, R.; Storm, R. *Tornado Cash* (2019).
9. Walras, L. *Éléments d’économie politique pure* (1874); Gnosis Protocol (2019); Penumbra ZSwap (2023).
10. Rubin, J. *BIP-119 (CTV)*; *OP_CAT* and *OP_CHECKSIGFROMSTACK* proposals.
11. Tacit. *Secret Sats* (2026), companion paper.